COMPUTER VISION

CAT 2 — Complete Open-Book Reference Notes

Module 3: Feature Extraction & Image Segmentation
Module 4: Pattern Analysis & Motion Analysis

Covers: Edge Detection · LoG/DoG · Hough Transform · Corner Detection · RANSAC · SIFT/SURF/HOG/GLOH · Scale-Space & Pyramids · Gabor Filters & DWT · Image Segmentation (all methods) · Object Detection & Evaluation · Pattern Analysis · Clustering (K-Means, K-Medoids, GMM) · PCA · LDA · Classifiers (Bayes, KNN, ANN) · Motion Analysis

Table of Contents

1. Feature Extraction — Overview
2. Edge Detection (Sobel, Prewitt, Canny)
3. LoG and DoG
4. Hough Transform
5. Corner Detection (Moravec, Harris, Hessian)
6. RANSAC
7. Feature Descriptors: SIFT
8. SURF, HOG, GLOH + Comparison
9. Scale-Space Analysis & Image Pyramids
10. Gabor Filters
11. Discrete Wavelet Transform (DWT)
12. Image Segmentation — Region & Edge Based
13. Split & Merge, Graph-Cut, Mean-Shift, MRF
14. Texture Segmentation (GLCM, LBP)
15. Object Detection & Evaluation Metrics
16. Pattern Analysis — Overview
17. Clustering: K-Means, K-Medoids
18. Gaussian Mixture Models (GMM / EM)
19. PCA — Principal Component Analysis
20. LDA — Linear Discriminant Analysis
21. Discriminant Functions
22. Classifiers: Bayes, KNN, ANN
23. Learning Paradigms & Classifier Evaluation
24. Motion Analysis (Background Subtraction, Optical Flow, KLT)
25. Master Formula Sheet & Exam Checklist

1. Feature Extraction — Overview

Why features? Raw pixel intensities are too high-dimensional, noisy and viewpoint-sensitive to compare images directly. Feature extraction converts an image into a compact set of numbers (edges, corners, blobs, texture, descriptor vectors) that are more robust to noise, scale, rotation and illumination, and can be used for matching, recognition, tracking and segmentation.

Overview of the feature extraction pipeline

Raw Image → **Pre-processing (smoothing)** → **Interest point / edge detection** → **Descriptor computation** → **Matching / Classification**

Categories of features covered in this module:

- **Edges** — Canny, LoG, DoG (1-D discontinuities / boundaries)
- **Lines** — Hough Transform (parametric shape detection)
- **Corners / interest points** — Harris, Hessian-Affine (2-D discontinuities, distinctive locations)
- **Local descriptors** — SIFT, SURF, HOG, GLOH (rich vectors around interest points, used for matching)
- **Multiscale/texture filters** — Gaussian pyramids, Gabor filters, DWT

2. Edge Detection

2.1 What is an edge?

An edge is a location in the image where intensity changes sharply — typically at object boundaries, surface-orientation changes, illumination changes, or depth discontinuities. Mathematically, edges occur where the **gradient magnitude is large**.

$$\nabla I = [I_x, I_y]^T = [\partial I/\partial x, \partial I/\partial y] \quad |\nabla I| = \sqrt{(I_x^2 + I_y^2)} \quad \theta = \tan^{-1}(I_y / I_x)$$

2.2 Gradient Operators

Operator	Gx kernel	Gy kernel	Notes
Roberts	[[1,0],[0,-1]]	[[0,1],[-1,0]]	2×2, very sensitive to noise
Prewitt	[[-1,0,1],[-1,0,1],[-1,0,1]]	[[-1,-1,-1],[0,0,0],[1,1,1]]	Simple averaging + differencing
Sobel	[[-1,0,1],[-2,0,2],[-1,0,1]]	[[-1,-2,-1],[0,0,0],[1,2,1]]	Weighted centre row/col — smoother, most common

Worked Numerical — Sobel Gradient at a Pixel

3×3 neighbourhood around the pixel of interest:

50	50	90
50	60	90
50	50	90

Apply Sobel Gx (element-wise multiply and sum):
Gx = (-1·50+0·50+1·90) + (-2·50+0·60+2·90) + (-1·50+0·50+1·90)
= (40) + (80) + (40) = **160**
Apply Sobel Gy:
Gy = (-1·50-2·50-1·90) + (0) + (1·50+2·50+1·90)
= (-190) + 0 + (190) = **0**
$|\nabla I| = \sqrt{(160^2+0^2)} = \mathbf{160}$; $\theta = \tan^{-1}(0/160) = \mathbf{0°}$ (a vertical edge, since intensity changes left→right only).
Exam method: always (1) write the 3×3 patch, (2) multiply term-by-term with Gx then Gy kernels and sum each, (3) compute magnitude & direction with the two formulas above.

2.3 Canny Edge Detector

The most widely used edge detector — optimizes 3 criteria: **good detection** (find all real edges), **good localization** (marked close to true edge), **single response** (one marking per edge).

1. Gaussian Smoothing → **2. Gradient (Sobel) Magnitude & Direction** → **3. Non-Max Suppression** → **4. Double Thresholding** → **5. Hysteresis Edge Tracking**

1. **Gaussian smoothing:** convolve with a Gaussian kernel to suppress noise before differentiating (differentiation amplifies noise).
2. **Gradient computation:** apply Sobel (or similar) to get magnitude M(x,y) and direction θ(x,y), quantized to 4 directions (0°,45°,90°,135°).
3. **Non-Maximum Suppression (NMS):** for each pixel, compare its magnitude with the two neighbours lying along the gradient direction θ. Keep the pixel only if its magnitude is the local maximum among these three; otherwise suppress it to 0. This thins wide edge responses down to 1-pixel width.
4. **Double thresholding:** using two thresholds T_low < T_high:
   - M ≥ T_high → **strong edge** (definitely kept)
   - T_low ≤ M < T_high → **weak edge** (kept only conditionally)
   - M < T_low → discarded (not an edge)
5. **Edge tracking by hysteresis:** a weak edge pixel is retained only if it is 8-connected to a strong edge pixel (directly or via a chain of weak edges leading to one); all other weak pixels are removed as noise.

Numerical illustration of thresholding

Suppose after NMS, a row of gradient magnitudes is: 5, 12, 55, 80, 60, 15, 4, with T_low=20, T_high=50.
5→discard, 12→discard, 55→**strong**, 80→**strong**, 60→**strong**, 15→discard, 4→discard.
If instead a pixel had magnitude 30 (weak) and is adjacent to the "55" strong pixel, it is **kept** (connected to strong); a weak pixel of 25 sitting isolated far from any strong pixel is **discarded**.

Advantages: low error rate, edges localized well, single response per edge. **Limitations:** performance depends heavily on choice of σ (Gaussian) and the two thresholds; computationally more expensive than simple Sobel.

3. LoG and DoG (Laplacian / Difference of Gaussian)

3.1 Laplacian of Gaussian (LoG)

Edges can also be found as **zero-crossings of the second derivative**. The Laplacian $\nabla^2 I = I_{xx}+I_{yy}$ is very noise-sensitive, so it is combined with Gaussian smoothing first:

$$LoG(x,y) = \nabla^2[G_\sigma(x,y) * I(x,y)] = [\nabla^2 G_\sigma] * I, \quad \nabla^2 G_\sigma(x,y) = [(x^2+y^2-2\sigma^2)/\sigma^4]\cdot\exp(-(x^2+y^2)/2\sigma^2)$$

Shape: a "Mexican-hat" (sombrero) function — positive centre lobe, negative surround. Zero-crossings of the LoG response mark edges. Larger σ → detects coarser, larger-scale edges/blobs; smaller σ → fine detail.

3.2 Difference of Gaussian (DoG)

DoG approximates the LoG much more cheaply, by simply subtracting two Gaussian-blurred versions of the same image at different scales σ1 < σ2:

```
g1(x,y)=Gσ1*f(x,y); g2(x,y)=Gσ2*f(x,y); DoG*f = g1−g2 = (Gσ1−Gσ2)*f(x,y)
```

```
DoG(x,y) ≜ Gσ1−Gσ2 = (1/√2π)·[ (1/σ1)e^(−(x²+y²)/2σ1²) − (1/σ2)e^(−(x²+y²)/2σ2²) ]
```

Why DoG≈LoG works: the Gaussian satisfies the heat-diffusion equation $\partial G/\partial\sigma = \sigma\nabla^2 G$, so a finite difference of two Gaussians at nearby scales is proportional to the scale-normalized Laplacian — DoG reuses already-blurred images (no extra 2nd-derivative convolution), which is why SIFT uses DoG instead of LoG for speed. **Edge-detection procedure using DoG:**

Apply DoG filter → **Find zero-crossings** → **Threshold zero-crossings (keep only strong swings)** → **Final edge map**

The threshold step suppresses weak zero-crossings likely caused by noise — only keep crossings where the difference between the adjacent positive maximum and negative minimum is large.

> **Conceptual numerical**
> If a 1-D DoG response along a scan-line is: ..., +0.02, +0.01, −0.01, −0.03, +0.02, ... — there is a sign change (zero-crossing) between the sample at +0.01 and the sample at −0.01. The swing magnitude there is $|0.01-(-0.01)|=0.02$ — if this is below the chosen threshold, discard as noise; if above, mark as an edge.

LoG vs DoG (exam favourite comparison):

Aspect	LoG	DoG
Computation	Direct 2nd derivative of Gaussian, convolved once	Difference of two separately Gaussian-blurred images (cheaper, reuses pyramids)
Accuracy	Exact	Close approximation, error small when σ1 close to σ2
Used in	Blob detection theory	SIFT keypoint detection (across scale-space)

4. Hough Transform

4.1 Motivation and Cartesian formulation

Used to detect parametric shapes (lines, circles) from a set of edge points even with noise/gaps, by a "voting" scheme in parameter space.

```
Line: y = mx + c  →  rearranged: c = (−x)m + y  (equation of a line in (m,c) parameter space; for a fixed image point (xᵢ,yᵢ): c = −xᵢ·m + yᵢ)
```

Each image point (x_i,y_i) becomes a *line* in (m,c) space; points that are collinear in the image produce lines in parameter space that all **intersect at one (m,c)** — that intersection accumulates votes = the detected line's parameters. Problem: slope m is unbounded for vertical lines.

4.2 Polar (Normal) Parametrization

```
ρ = x·cosθ + y·sinθ
```

ρ = perpendicular distance from the origin to the line; θ = angle the line's normal makes with the x-axis. For a given line, ρ≥0 and 0≤θ≤360° are always bounded — this solves the vertical-line problem.

4.3 Algorithm (Voting Procedure)

Detect edge pixels → **For each pixel and every candidate θ, compute ρ=x cosθ+y sinθ** → **Increment accumulator A(ρ,θ)** → **Find peak(s) in accumulator** → **Peak (ρ,θ) = detected line**

"N samples are needed to fit a model (2 points define a line), but even one sample brings information — it votes for all models (lines) consistent with it. Collecting votes from all points and seeking consensus (the highest-vote cell) identifies the dominant line(s)."

> **Worked Numerical 1 — 3 collinear points**
> Points: (10,10), (20,20), (30,30) — all lie on y=x.
> Try θ = 135° ($\cos135°=-0.707$, $\sin135°=0.707$):
> $\rho_1 = 10(-0.707)+10(0.707) = 0$
> $\rho_2 = 20(-0.707)+20(0.707) = 0$
> $\rho_3 = 30(-0.707)+30(0.707) = 0$
> All three points vote for the SAME cell (ρ=0, θ=135°) → 3 votes → this is the accumulator peak.
> **Detected line:** ρ=0, θ=135° corresponds to y=x (slope $m=-\cos\theta/\sin\theta = -(-0.707)/0.707 = 1$, intercept c=0).

Worked Numerical 2 — Accumulator table for 3 points

Points: A(0,0), B(4,4), C(8,8). Build the accumulator for θ ∈ {0°, 45°, 90°, 135°}:

θ	ρ for A(0,0)	ρ for B(4,4)	ρ for C(8,8)	Votes at same ρ
0°	0	4	8	1 each (no match)
45°	0	5.66	11.31	1 each (no match)
90°	0	4	8	1 each (no match)
135°	0	0	0	**3 votes at ρ=0** ← peak

Maximum votes (3) occur at (ρ=0, θ=135°) → detected line is y=x, matching all points exactly. *General exam method: for each given point and each candidate θ, compute ρ=x cosθ+y sinθ, tabulate as above, find the (ρ,θ) cell where all/most points agree — that is the fitted line; recover slope/intercept if asked using m=−cosθ/sinθ and solving c from y=mx+c using any one point.*

5. Corner Detection

5.1 Why corners?

Corners are formed where two edges/regions meet and the boundary curvature is high, showing **large intensity variation in all directions** (unlike an edge, which varies in only one direction). Corners are stable, repeatable "interest points" / landmark points used for tracking, matching, and stereo correspondence.

5.2 Moravec Corner Detector (older/traditional)

- Places a small window at each pixel and shifts it in several discrete directions (0°,45°,90°,...)
- Computes the sum-of-squared-differences (SSD) "edge strength" for each shift direction
- Takes the **minimum** SSD across all directions as the corner response (a true corner has a large minimum, since it changes a lot in every direction)
- Thresholds this response and applies non-maximum suppression to extract discrete corner points

Limitations: not rotation invariant (only tests a discrete set of directions), sensitive to noise/small variations in intensity.

5.3 Harris Corner Detector

Improves on Moravec using a continuous, differentiable formulation via the auto-correlation/structure-tensor matrix.

```
E(u,v) ≈ [u,v]·M·[u,v]ᵀ,   M = Σw(x,y)·[[Ix², IxIy],[IxIy, Iy²]]
```

M is built from local gradients (via Sobel or similar), weighted by a window function w (often Gaussian). Eigen-analysis of M (eigenvalues λ1, λ2) tells us the local structure:

- λ1, λ2 both small → **flat region** (no structure)
- one large, one small → **edge** (varies in only one direction)
- both λ1, λ2 large → **corner** (varies strongly in all directions)

To avoid expensive eigenvalue computation, Harris defines a response function using only det and trace of M:

```
R = det(M) − k·(trace(M))², where det(M)=λ1λ2=ac−b², trace(M)=λ1+λ2=a+c, k≈0.04–0.06
```

Interpretation of R: R large and positive → corner; R large negative → edge; |R| small → flat region.

Worked Numerical — Harris Response

Given at a pixel: ΣIx²=4, ΣIy²=6, ΣIxIy=2 (so a=4, c=6, b=2).
M = [[4,2],[2,6]].
det(M) = (4)(6) − (2)² = 24 − 4 = **20**
trace(M) = 4 + 6 = **10**
Using k=0.04: R = 20 − 0.04(10²) = 20 − 4 = **16**
R is positive and reasonably large ⇒ **corner region**.
*If det(M) had instead been small relative to trace² (e.g. det=2, trace=10 → R=2−4=−2, negative) that indicates an **edge**. If both det and trace are small (e.g. det=0.1, trace=0.5 → R≈0.09) that indicates a **flat** region. Template: build M from Ix²,Iy²,IxIy → det=ac−b² → trace=a+c → R=det−k·trace² → classify by sign/magnitude.*

5.4 Hessian / Hessian-Affine Detector

```
H = [[Ixx, Ixy],[Ixy, Iyy]],   det(H) = Ixx·Iyy − Ixy²
```

Large |det(H)| indicates a blob-like structure (used for blob/interest-point detection, complementary to Harris' corner focus). The **Hessian-Affine** extension fits an ellipse (via the second-moment matrix) to each detected point to normalize for affine (viewpoint) distortion, giving better repeatability under viewpoint change than plain Harris, though on its own it is not scale-invariant without embedding in a scale-space search.

6. RANSAC (Random Sample Consensus)

An iterative, robust model-fitting method used when the data contains **outliers** that would corrupt a standard least-squares fit.

1. Randomly select minimal subset → **2. Fit model to subset** → **3. Count inliers (points close to model)** → **4. Repeat for N iterations** → **5. Keep model with most inliers**

For fitting a line y=mx+c, the minimal subset is just 2 points. After proposing a line from 2 random points, RANSAC counts how many of the remaining points lie within a small distance tolerance of that line (inliers); the process repeats and the line with the largest inlier count is chosen as the final robust fit — outliers are automatically ignored because they rarely agree with any single good model. **Applications in computer vision:**

- Estimating the fundamental/essential matrix in stereo vision (robust to mismatched correspondences)
- Finding correspondences between two sets of feature points for feature-based object detection/matching
- Registering (aligning) sequential video frames for video stabilization

> **Conceptual Numerical**
> Suppose 10 points are mostly near the line y=2x+1 but 3 are random outliers. RANSAC randomly picks 2 points, fits a candidate line, and checks how many of the other 8 points fall within, say, ±2 units of that line. A candidate line fit from two "good" points will typically get ~7 inliers (out of 10), while a line fit using an outlier point will usually get very few inliers (~2). After many trials, RANSAC keeps the line with the highest inlier count (~7/10) as the robust estimate — very close to the true underlying line y=2x+1, ignoring the 3 outliers.

7. Feature Descriptors — SIFT (Scale-Invariant Feature Transform)

SIFT extracts distinctive keypoints that are invariant to scale, rotation, and (partially) illumination/affine change — foundational for matching, panorama stitching, object recognition.

7.1 Full Pipeline

1. Scale-space construction → **2. DoG extrema detection** → **3. Keypoint localization & filtering** → **4. Orientation assignment** → **5. Descriptor construction (128-D)**

1. **Scale-space construction:** repeatedly Gaussian-blur the image at increasing σ, organized into "octaves" (each octave = image downsampled by 2× from previous); build a DoG pyramid by subtracting adjacent blur levels within each octave.
2. **DoG extrema (keypoint candidates):** compare each sample point to its **26 neighbours** — 8 in the same scale, 9 in the scale above, 9 in the scale below. If it is a local minimum or maximum among all 26, it's a candidate keypoint (this searches for extrema jointly in space AND scale).
3. **Keypoint localization/filtering:** refine the location using a Taylor expansion of the DoG function (sub-pixel accuracy); reject **low-contrast** points (small DoG response — likely noise) and points lying along an **edge** (checked via the ratio of eigenvalues of the 2×2 Hessian at that point — similar spirit to Harris; edges have one large, one small eigenvalue, so are rejected as unstable keypoints).
4. **Orientation assignment:** build a histogram of local gradient directions (weighted by magnitude and a Gaussian window) around the keypoint; the dominant peak direction becomes the keypoint's canonical orientation — this is what makes SIFT **rotation-invariant** (the descriptor is later computed relative to this orientation).
5. **Descriptor construction:** take a 16×16 pixel patch around the keypoint (rotated to the canonical orientation), divide into 4×4=16 sub-blocks, compute an 8-bin orientation histogram in each sub-block → 16×8 = **128-dimensional vector**. Normalize to unit length (illumination-invariance), clip large values (reduce sensitivity to non-linear illumination changes / camera saturation), and renormalize.

> **Worked Numerical — Gradient magnitude/orientation feeding into SIFT/HOG bins**
> At a pixel, Ix=3, Iy=4.
> Magnitude = $\sqrt{(3^2+4^2)} = \sqrt{25} =$ **5**
> Orientation = $\tan^{-1}(4/3) =$ **53.13°**
> If this pixel's Gaussian-window weight is w=0.8, its contribution (vote) into the matching orientation histogram bin = w × magnitude = 0.8 × 5 = **4.0** (in practice this is split between the two nearest orientation bins by bilinear interpolation based on angular distance to each bin centre — e.g. if bins are centred at 45° and 67.5°, most of the 4.0 vote goes to the 45° bin since 53.13° is closer to it).

Why 128-D and normalization matter: the 128 numbers make the descriptor highly distinctive (low false-match rate) while normalization + clipping give strong robustness to illumination changes — this is why SIFT is a common default choice in matching-heavy or accuracy-critical tasks.

8. SURF, HOG, GLOH and Comparison

8.1 SURF (Speeded-Up Robust Features)

Designed as a much faster approximation to SIFT.

- **Detector:** approximates the Hessian determinant using **box filters** computed via the **integral image** $II(x,y)=\Sigma_{i\le x,j\le y} I(i,j)$, so any rectangular sum is computed in O(1) time regardless of size.
- Approx.: $\det(H_{approx}) \approx D_{xx} \cdot D_{yy} - (0.9 \cdot D_{xy})^2$

- **Descriptor:** uses Haar-wavelet responses in x and y within sub-regions around the keypoint → typically a **64-D** vector (an "extended" 128-D version also exists).
- **Speed:** roughly 3× faster than SIFT due to integral-image tricks, at some cost in matching accuracy/robustness (weaker under strong illumination or viewpoint change than SIFT).

8.2 HOG (Histogram of Oriented Gradients)

Unlike SIFT/SURF/GLOH, HOG does **not** use interest-point detection — it is computed **densely** over the whole image or window (e.g. a pedestrian bounding box).

Compute gradient (Gx,Gy) at every pixel → **Magnitude & orientation per pixel** →
Orientation histogram per cell (e.g. 8×8 px, 9 bins over 0–180°) → **Group cells into overlapping blocks** → **Block normalization** →
Concatenate = final HOG vector

```
L2 normalization: v' = v / √(‖v‖² + ε²)  (or L2-Hys: L2-normalize → clip large values → renormalize)
```

HOG captures the overall **shape/silhouette** of an object via local edge-orientation distribution — this makes it particularly effective for pedestrian and rigid-object detection, but it is comparatively weak to scale changes unless run over an image pyramid (multi-scale sliding window).

8.3 GLOH (Gradient Location and Orientation Histogram)

An extension of SIFT designed to be more distinctive by changing the spatial binning from a square grid to a **log-polar** grid.

- Spatial bins: 3 radial × ~8 angular divisions (commonly cited as 17 spatial locations) × 16 orientation bins = 272-D raw histogram.
- **PCA** is applied to reduce 272-D down to a more manageable **128-D** vector (same size as SIFT, but typically more discriminative).
- Log-polar binning (radius $r=\sqrt{(x^2+y^2)}$, angle $\varphi=\tan^{-1}(y/x)$) captures spatial layout more naturally for many textures/objects than SIFT's square Cartesian grid.

8.4 Comparison Table

Descriptor	Keypoint Detection	Description Method	Dimension	Scale/Rotation Invariance	Computational Cost	Robustness
SIFT	DoG extrema across scale-space	4×4 grid × 8 orientation bins	128	Strong (both)	High	Very robust to illumination/viewpoint; the classic accuracy baseline
SURF	Fast-Hessian via box filters + integral image	Haar-wavelet responses	64 (or 128 extended)	Strong (both)	Low (fast, ~3× SIFT speed)	Good but somewhat less robust than SIFT under strong changes
HOG	None — dense sliding window/grid	Orientation histograms, block-normalized	Varies with window size	Weak (no built-in scale/rotation handling)	Moderate	Excellent for shape/silhouette (e.g. pedestrians); weak to rotation
GLOH	Same detector as SIFT (DoG)	Log-polar bins → PCA to 128-D	128 (after PCA)	Strong (both), generally more distinctive than SIFT	Higher than SIFT (extra PCA step)	Slightly better matching accuracy than SIFT in many benchmarks

8.5 Which descriptor for medical image analysis?

Recommendation: SIFT (or GLOH if compute budget allows).

- Medical images vary in **scale/zoom** (different acquisition devices/settings) and **orientation** (patient positioning) — both SIFT and GLOH are explicitly scale- and rotation-invariant, unlike HOG.
- Medical applications (image registration for diagnosis, tumor tracking across scans) prioritize **matching accuracy over speed** — SIFT's illumination-robust normalization and high distinctiveness reduce the risk of false matches, which is critical when a mismatch could mean misregistering anatomy.
- SURF would be preferred only if **real-time speed** were the priority (e.g. live ultrasound tracking), trading off some accuracy.
- GLOH can be even more distinctive than SIFT (useful for subtle texture differences in tissue), but its extra PCA computation adds cost — a reasonable justified choice if accuracy is paramount and speed is not constrained.

9. Scale-Space Analysis and Image Pyramids

9.1 Why scale-space?

Real-world structures exist at different sizes — a small "edge" detector tuned to one resolution will miss coarser structures and vice versa. Scale-space analysis represents the image at a continuum of scales, letting later processing choose (or search

across) the "right" scale.

```
L(x,y,σ) = Gσ(x,y) * I(x,y),   Gσ(x,y) = (1/2πσ²)·exp(−(x²+y²)/2σ²)
```

Semi-group property: Gσ1 * Gσ2 = G√(σ1²+σ2²) — blurring twice with σ1 then σ2 is equivalent to blurring once with √(σ1²+σ2²). As σ increases, fine detail is progressively suppressed and only coarse structure remains. **Scale-normalized Laplacian** σ²∇²L is used to find "characteristic scale" of blob-like structures — the scale at which the response is maximized indicates the natural size of that blob (this underlies SIFT's search for extrema across scale).

9.2 Image Pyramids

Gaussian Pyramid

Built by repeatedly blurring and downsampling by a factor of 2:

```
G_{l+1}(x,y) = Σ_m Σ_n w(m,n)·G_l(2x+m, 2y+n)
```

Each level is half the width/height of the previous, giving a multi-resolution representation (level 0 = original image, higher levels = coarser).

Laplacian Pyramid

Captures the detail (high-frequency information) *lost* between Gaussian levels:

```
L_l = G_l − Expand(G_{l+1})
```

where Expand upsamples G_{l+1} back to G_l's resolution (e.g. via interpolation) before subtracting. The original image can be exactly reconstructed by summing all Laplacian levels plus the coarsest Gaussian level. **Applications of pyramids:** coarse-to-fine search/optimization (e.g. optical flow, stereo matching — solve at coarse scale first, refine at finer scales), efficient multiscale object/feature search, seamless image blending (Laplacian pyramid blending), data compression.

9.3 Gaussian Derivative Filters

Instead of differentiating a noisy raw image, differentiate the Gaussian kernel analytically and convolve once (equivalent to smoothing+differentiating together, mathematically identical, more efficient):

```
1st order: Gx = −(x/σ²)·Gσ(x,y); Gy = −(y/σ²)·Gσ(x,y)
```

```
2nd order: Gxx = [(x²/σ⁴) − (1/σ²)]·Gσ(x,y) (similarly Gyy, Gxy — together form the Hessian matrix)
```

These filters are used throughout: Sobel/Canny effectively approximate first-order Gaussian derivatives; the Harris/Hessian corner detectors use exactly these derivative responses; LoG is the trace of the second-order Gaussian-derivative Hessian.

10. Gabor Filters

Gabor filters are sinusoidal gratings modulated by a Gaussian envelope — jointly localized in **both space and frequency** (mathematically optimal in the space-frequency trade-off, per the Heisenberg uncertainty principle), making them excellent texture/orientation feature extractors.

```
g(x,y) = exp[−(x'² + γ²y'²)/(2σ²)] · cos(2πx'/λ + ψ)
```

```
x' = x·cosθ + y·sinθ,   y' = −x·sinθ + y·cosθ
```

Parameters:

- λ — wavelength of the sinusoidal carrier (controls spatial frequency selectivity)
- θ — orientation of the filter (which direction of texture/edges it responds to)
- ψ — phase offset (symmetric ψ=0 vs anti-symmetric ψ=90° filters)
- σ — standard deviation of the Gaussian envelope (controls spatial extent / bandwidth)
- γ — spatial aspect ratio (ellipticity of the support region)

Gabor filter bank: in practice, a set of Gabor filters at multiple (λ,θ) combinations is applied to the image; the collection of responses at each pixel forms a rich multi-orientation, multi-frequency feature vector, well-suited to capturing texture. **Applications:** texture classification, face recognition, iris/fingerprint recognition, edge/line detection at specific orientations.

> **Conceptual numerical**
> A Gabor filter with θ=0° responds strongly to vertical stripes (since x' aligns with x, the cosine grating oscillates along x — i.e., it detects intensity variation along the horizontal direction, which corresponds to vertical edges/stripes). A filter with θ=90° would instead respond strongly to horizontal stripes. A texture with strong 45° diagonal patterns will give the highest filter-bank response at θ=45°, close to zero response at θ=135° (orthogonal orientation) — this pattern of responses across θ is itself a texture "signature" usable for classification.

11. Discrete Wavelet Transform (DWT)

DWT decomposes a signal/image into **approximation** (low-frequency) and **detail** (high-frequency) components at multiple resolutions, using a pair of filters and downsampling.

```
Approximation: a[k] = Σ_n h[n−2k]·x[n]  (low-pass filter h, then downsample by 2)
```

```
Detail: d[k] = Σ_n g[n−2k]·x[n]  (high-pass filter g, then downsample by 2)
```

11.1 2-D DWT on Images

Applying 1-D DWT along rows then columns produces **4 subbands** per level:

- **LL** — low-pass in both directions = coarse approximation (looks like a smaller, smoothed version of the image)
- **LH** — low-pass rows, high-pass columns = captures horizontal edges/detail
- **HL** — high-pass rows, low-pass columns = captures vertical edges/detail
- **HH** — high-pass in both = captures diagonal detail

Recursively applying DWT to the LL subband gives a **multi-level (multi-resolution) decomposition** — e.g. Level-2 LL is a further-smoothed, further-downsampled approximation, with its own LH/HL/HH detail bands.

11.2 Texture Features from Wavelet Subbands

```
Energy: E = Σ|C(i,j)|²     Entropy: H = −Σ p(i,j)·log₂ p(i,j)
```

where C are the subband coefficients and p is the normalized coefficient-magnitude distribution. High energy in a particular subband (e.g., HL) suggests strong directional texture content (vertical edges) in that region.

Worked Numerical — DWT Feature Computation
Detail sub-band coefficients: [2, −1, 3, 0]
Energy = $2^2 + (-1)^2 + 3^2 + 0^2 = 4+1+9+0 =$ **14**
Absolute sum (a simpler texture measure) = |2|+|−1|+|3|+|0| = **6**
Mean absolute value = 6/4 = **1.5** — used as a compact per-subband texture descriptor for classification.

11.3 Gabor vs DWT — Comparison

Aspect	Gabor Filters	DWT
Space-frequency localization	Optimal joint localization (tunable, continuous θ and λ)	Good multi-resolution localization, but only axis-aligned (≈0°/45°/90°) directions per level
Texture representation	Rich, directionally tunable — best when texture has specific dominant orientations	Good for general multi-scale texture/edge energy; less flexible in orientation
Computational cost	Higher — needs a bank of filters at multiple (θ,λ) convolved separately	Lower — separable filters + downsampling, very efficient (fast pyramidal algorithm)
Redundancy	Redundant (filter bank responses overlap in information)	Orthogonal/non-redundant (critical sampling — same amount of data before/after transform)
Typical applications	Texture/biometrics where dominant orientation matters (fingerprints, iris)	Compression (JPEG2000), denoising, fast multiresolution texture/edge features

12. Image Segmentation — Region and Edge Based Methods

12.1 What is segmentation?

Partitioning an image into meaningful regions where pixels within a region are similar and boundaries separate dissimilar regions. Applications: medical imaging (organ/tumor delineation), remote sensing, autonomous driving, industrial inspection.
Output representations: binary mask, multi-class semantic map, instance masks, or bounding boxes with class labels.

12.2 Region Growing

A bottom-up, seed-based method.

Select seed pixel(s) → **Inspect neighbours (4- or 8-connectivity)** → **Add neighbour if similarity criterion satisfied** → **Repeat until no new pixel qualifies**

```
Similarity test: |I(p) − μ_R| ≤ T  (I(p)=intensity at candidate pixel p, μ_R=current mean of region R, T=threshold)
```

Worked Numerical — Region Growing
5×5 image, seed = centre pixel with value 90, T=15:

20	20	20	20	20
20	80	80	80	20
20	80	90	80	20
20	80	80	80	20

20	20	20	20	20

Region mean starts at μ_R=90 (just the seed). Check the 8 neighbours of the seed: each has value 80 → |80−90|=10 ≤ 15 → **included**. Update μ_R = mean(90, 80×8) = 81.1. Now check the next ring (value 20): |20−81.1|=61.1 > 15 → **excluded**. Final region = the inner 3×3 block of values {90, 80×8} — correctly segments the bright object from the dark background.

Strengths: simple, produces connected regions — good for medical images with a clear seed. **Limitations:** highly sensitive to seed choice and threshold T; can leak across weak boundaries.

12.3 Split and Merge (Quadtree Segmentation)

A top-down, "divide and conquer" approach — the opposite direction from region growing.

Whole image = 1 region → **Homogeneous? If NO, split into 4 quadrants** → **Repeat recursively per quadrant (Phase I: Split)** → **Merge adjacent regions with similar properties (Phase II: Merge)** → **Final segmented regions**

Homogeneity test is often a simple criterion like max−min intensity within the region ≤ threshold, or variance ≤ threshold. Splitting stops when a region is homogeneous or has reached a minimum allowed size (e.g. 2×2). After splitting, adjacent leaf regions (even from different branches of the quadtree) that are similar are merged to avoid unnecessary blockiness. **Strengths:** systematic, no seed needed, guaranteed to terminate. **Limitations:** produces blocky, square-aligned boundaries; result can depend on where the quadtree splits fall relative to actual object boundaries.

12.4 Edge-Based Segmentation

Detect edges (Sobel/Canny, see Section 2) and treat them as region boundaries; regions are then the areas enclosed by closed edge contours (may need edge-linking/morphological closing to handle gaps). **Strengths:** good, precise boundaries when edges are strong. **Limitations:** broken/noisy edges in low-contrast or noisy images lead to incomplete boundaries and leaking regions.

13. Split & Merge Alternatives: Graph-Cut, Mean-Shift, MRF

13.1 Graph-Cut Segmentation

Formulates segmentation as global **energy minimization** on a graph.

- Represent each pixel (or superpixel) as a graph node.
- Edges between neighbouring pixels encode similarity (smoothness).
- Two special terminal nodes, source (S) and sink (T), represent the two competing labels (e.g. foreground/background).
- The **minimum cut** that separates S from T (via max-flow/min-cut algorithms) gives the globally optimal label assignment for the chosen energy.

```
E(L) = Σ_p D_p(L_p) + λ·Σ_{(p,q)∈N} w_pq·[L_p ≠ L_q]
```

- **D_p** — data term: cost of assigning label L_p to pixel p (e.g. how well it matches foreground/background colour model)
- **Smoothness term:** penalizes neighbouring pixels p,q being assigned different labels, weighted by w_pq (often based on how similar p and q are — encourages boundaries to fall where contrast is high)
- **λ** — balances data fidelity against boundary smoothness

Applications: interactive segmentation (e.g. GrabCut-style foreground extraction with user scribbles). **Limitation:** requires careful energy design (data + smoothness terms).

13.2 Mean-Shift Segmentation

A non-parametric, mode-seeking clustering method in a joint spatial+colour feature space.

Feature space (spatial+colour) → **Kernel density estimate** → **Mean-shift vector (climb toward density peak)** → **Convergence to a mode** → **Merge points converging to same mode** → **Final segments**

```
x_{t+1} = [ Σᵢ xᵢ·K(‖x−xᵢ‖²/h²) ] / [ Σᵢ K(‖x−xᵢ‖²/h²) ]
```

Each point iteratively shifts toward the local density maximum (mode); pixels that converge to the same mode form one cluster/segment. **Bandwidth h** is the key parameter: small h → many small clusters (over-segmentation); large h → fewer, smoother regions (possible under-segmentation/merging). A major advantage: **no need to pre-specify the number of clusters** (unlike K-Means).

13.3 Markov Random Fields (MRF)

Models segmentation as assigning a **hidden label** to every pixel, where neighbouring labels statistically influence each other (the Markov property: a pixel's label depends only on its local neighbourhood, not the whole image directly).

```
E(L) = Σ_p ψ(L_p) + β·Σ_{(p,q)∈N} ψ(L_p, L_q)
```

The first term is a unary "data" potential (per-pixel evidence for a label); the second is a pairwise "smoothness" potential (clique potential) between neighbours, weighted by β. The final segmentation is obtained via **MAP (Maximum A Posteriori) estimation** — finding the label configuration L that minimizes this energy (equivalently maximizes the posterior probability).

Optimization methods: Iterated Conditional Modes (ICM — greedy, fast, can get stuck in local minima), graph-based optimization (for certain energy forms, exact), belief propagation, simulated annealing / Gibbs sampling (slower, but can escape local minima). **Strength:** naturally models spatial context/neighbourhood coherence, good for noisy images. **Limitation:** computationally expensive, especially with iterative sampling-based solvers.

13.4 Segmentation Method Comparison (Master Table)

Method	Main Principle	Strength	Limitation	Typical Use
Region Growing	Local similarity, seed-based	Simple, connected regions	Sensitive to seed/threshold	Medical regions
Split & Merge	Quadtree homogeneity test	Systematic, no seed needed	Blocky artifacts	General-purpose, structured scenes
Edge-Based	Intensity gradients	Good, precise boundaries	Broken/noisy edges	Object boundaries
Graph-Cut	Global energy minimization	Global optimum	Energy design required	Interactive segmentation
Mean-Shift	Density mode-seeking	No fixed cluster count	Bandwidth sensitive	Colour segmentation
MRF	Spatial probability + MAP	Models neighbourhood context	Computational cost	Noisy images
Texture Segmentation	Pattern/GLCM features	Separates similar-intensity regions	Requires feature selection	Material/tissue analysis

14. Texture Segmentation (GLCM, LBP, and Related Features)

When regions have similar average intensity but different **patterns** (texture), intensity-based methods fail — texture features are needed.

14.1 Gray-Level Co-occurrence Matrix (GLCM)

GLCM P(i,j) counts how often a pixel with intensity i occurs adjacent (at a given offset/direction) to a pixel with intensity j. From this matrix, texture descriptors are computed:

```
Contrast = Σᵢ Σⱼ (i−j)² · P(i,j)
```

```
Homogeneity = Σᵢ Σⱼ P(i,j) / (1 + |i−j|)
```

```
Energy = Σᵢ Σⱼ P(i,j)²     Correlation = Σᵢ Σⱼ (i−μᵢ)(j−μⱼ)P(i,j) / (σᵢσⱼ)
```

Worked Numerical — GLCM features
Given a (simplified, already-normalized) 2×2 GLCM: P(0,0)=0.5, P(0,1)=0.2, P(1,0)=0.2, P(1,1)=0.1
Contrast = $(0-0)^2(0.5)+(0-1)^2(0.2)+(1-0)^2(0.2)+(1-1)^2(0.1)$ = 0+0.2+0.2+0 = **0.4**
Homogeneity = 0.5/(1+0) + 0.2/(1+1) + 0.2/(1+1) + 0.1/(1+0) = 0.5+0.1+0.1+0.1 = **0.8**
Energy = $0.5^2+0.2^2+0.2^2+0.1^2$ = 0.25+0.04+0.04+0.01 = **0.34**
High homogeneity + high energy here indicates a fairly uniform, low-contrast texture patch.

14.2 Other Texture Features

- **Local Binary Patterns (LBP):** for each pixel, threshold its circular neighbours against the centre pixel's value (1 if neighbour ≥ centre, else 0), forming a binary code converted to a decimal "LBP value"; a histogram of LBP values over a region is the texture descriptor. Simple, fast, illumination-invariant (relative comparisons only).
- **Gabor filter responses** (Section 10) — filter-bank energy per orientation/frequency as texture features.
- **Wavelet/filter-bank features** (Section 11) — subband energy/entropy as texture features.
- **Deep texture features** — CNN-based descriptors (mentioned for completeness; less exam-numerical, more conceptual).

14.3 Texture Segmentation Pipeline

Image → **Compute texture features (GLCM/LBP/Gabor)** → **Feature vector per pixel/patch** → **Classifier or clustering (e.g. K-Means)** → **Region labels**

Well-suited to material/tissue analysis, where regions have the same average brightness but structurally different patterns (e.g. different tissue types in an MRI slice).

15. Object Detection and Evaluation Metrics

15.1 Object Detection Output and Pipelines

Output per detected object: **class label**, **bounding box** B=(x,y,w,h), **confidence score**, optionally an **instance mask**.

Pipeline	Steps
Traditional	Sliding windows → handcrafted features (HOG, Haar) → classifier (SVM/AdaBoost) → Non-Max Suppression
Deep Learning — Two-stage	Region proposals → CNN feature extraction → classification + box regression (e.g. Faster R-CNN)
Deep Learning — One-stage	Single CNN pass predicts boxes + classes directly, uses feature pyramids for multi-scale detection (e.g. YOLO, SSD)

15.2 Intersection over Union (IoU)

```
IoU = Area(Intersection) / Area(Union)
```

IoU close to 1 → strong localization; IoU close to 0 → poor overlap. A detection is usually considered a "true positive" only if its IoU with the ground-truth box exceeds a chosen threshold (commonly 0.5).

> **Worked Numerical — IoU**
> Two boxes have intersection area = 40 and union area = 100.
> IoU = 40/100 = **0.40**. If the threshold for a valid detection is 0.5, this detection would be counted as a **false positive** (insufficient overlap) despite being "close".

15.3 Precision, Recall, F1, mAP

```
Precision = TP/(TP+FP)    Recall = TP/(TP+FN)    F1 = 2·P·R/(P+R)
```

```
AP = area under the Precision-Recall curve for one class; mAP = mean AP across all classes
```

- High precision → few false positives (detector rarely wrong when it fires)
- High recall → few missed objects (detector rarely misses a true object)
- Use multiple metrics together — accuracy alone is misleading with imbalanced classes

> **Worked Numerical — Precision/Recall/F1**
> Given TP=80, FP=20, FN=10:
> Precision = 80/(80+20) = 80/100 = **0.80**
> Recall = 80/(80+10) = 80/90 ≈ **0.889**
> F1 = 2(0.80)(0.889)/(0.80+0.889) = 1.422/1.689 ≈ **0.842**

15.4 Segmentation-Specific Metrics

```
IoU/Jaccard = |A∩B| / |A∪B|; Dice = 2|A∩B| / (|A|+|B|); Pixel Accuracy = (correct pixels)/(total pixels)
```

15.5 Clustering Evaluation (Relevant Cross-Reference)

Within-cluster sum of squares (WCSS), Silhouette coefficient, Adjusted Rand Index (needs reference/ground-truth labels), Normalized Mutual Information — used to evaluate segmentation/clustering quality when comparing against ground truth or judging internal cluster compactness.

16. Pattern Analysis — Overview

Pattern: a measurable representation of an object or event — e.g. colour, shape, texture, edge, or motion descriptors — represented as a **feature vector**:

```
x = [x₁, x₂, ..., x_d]ᵀ ∈ ℝᵈ
```

Main Tasks in Pattern Analysis

- **Clustering** — discover natural groups in unlabeled data
- **Classification** — assign a known class label to a pattern
- **Density estimation** — model the underlying probability distribution of the data
- **Decision making** — select the most likely label given evidence

Typical Pipeline

Image/Signal → **Feature extraction** → **Pre-processing / normalization** → **Clustering or Classifier** → **Prediction + Evaluation**

17. Clustering: K-Means and K-Medoids

17.1 Clustering — General Idea

Unsupervised grouping: no class labels are given during learning; objects within a cluster should be similar, different clusters should be well separated. Common methods: K-Means, K-Medoids, Gaussian Mixture Models.

> Objective (general): $\min \Sigma_k \Sigma_{x_i \in C_k} \|x_i - \mu_k\|^2$ (minimize within-cluster variation; K = number of clusters, chosen in advance)

17.2 K-Means Clustering

Initialize K centroids → **Assign each point to nearest centroid** → **Update centroid = mean of assigned points** → **Repeat until stable (no reassignment)**

$$\mu_k = (1/|C_k|)\cdot\Sigma_{x_i \in C_k} x_i \qquad C_k = \{x_i : \|x_i-\mu_k\| \le \|x_i-\mu_j\| \ \forall j\}$$

Worked Numerical — Full K-Means Iteration
Data = {2, 3, 4, 10, 11, 12}, K=2, initial centroids μ1=3, μ2=11.
Step 1 — Assign: distance of each point to (μ1,μ2):
2 → (1, 9) → closer to μ1 → C1
3 → (0, 8) → C1
4 → (1, 7) → C1
10 → (7, 1) → C2
11 → (8, 0) → C2
12 → (9, 1) → C2
Step 2 — Update: μ1(new) = (2+3+4)/3 = **3**; μ2(new) = (10+11+12)/3 = **11**
Step 3 — Check convergence: centroids unchanged from initial values ⇒ **algorithm has converged** after just 1 iteration.
Final clusters: C1={2,3,4}, C2={10,11,12}.

Advantages: simple, fast, works well for compact/well-separated (roughly spherical) clusters, easy to implement and explain. **Limitations:** must choose K in advance; sensitive to initialization (different starting centroids can converge to different local optima); sensitive to outliers (mean is pulled by extreme values); assumes roughly spherical, similarly-sized clusters. **Choosing K:** run with multiple random initializations, compare final within-cluster sum of squares; use the **elbow method** (plot WCSS vs K, look for the "elbow" where improvement slows) or **silhouette analysis**.

17.3 K-Medoids

Similar to K-Means, but the cluster "center" (medoid) must be an **actual data point**, not an arithmetic mean.

$$\text{Cost} = \Sigma_i \min_j d(x_i, m_j) \quad (m_j = \text{medoids})$$

Algorithm (PAM — Partitioning Around Medoids): assign each point to its nearest medoid; try swapping each medoid with a non-medoid point, keep the swap if it reduces total cost; repeat until no improving swap exists.

Aspect	K-Means	K-Medoids
Center	Mean (may not be an actual data point)	Medoid (an actual observed data point)
Robustness to outliers	Low (mean is skewed by outliers)	High (medoid is a real, "typical" point)
Computational cost	Lower	Higher (evaluating swaps is expensive)

18. Gaussian Mixture Models (GMM) and the EM Algorithm

GMM is a **probabilistic**, "soft" clustering model — each cluster is a Gaussian component, and data is modeled as a weighted mixture of these Gaussians.

$$p(x) = \Sigma_{k=1}^{K} \pi_k \cdot \mathcal{N}(x \mid \mu_k, \Sigma_k), \quad \pi_k \ge 0, \ \Sigma\pi_k = 1$$

- π_k — mixing weight (prior probability) of component k
- μ_k — mean of Gaussian component k
- Σ_k — covariance matrix of component k (allows elliptical, not just spherical, clusters)

Soft assignment (responsibility):

$$\gamma_k(x) = \pi_k \cdot \mathcal{N}(x|\mu_k,\Sigma_k) \ / \ \Sigma_j \ \pi_j \cdot \mathcal{N}(x|\mu_j,\Sigma_j)$$

18.1 EM (Expectation-Maximization) Algorithm

Initialize π, μ, Σ → **E-step: compute responsibilities $\gamma_k(x)$ for every point** → **M-step: update π, μ, Σ using those responsibilities** → **Repeat until log-likelihood converges**

Worked Concept — Soft Assignment
For a point x: γ1(x)=0.70, γ2(x)=0.30 → this means x is **70% associated with component 1** and 30% with component 2 (note $\Sigma\gamma_k(x)=1$ always).
Contrast with K-Means: in K-Means, this point would be assigned **entirely** to whichever single cluster is nearest (hard assignment); GMM instead keeps a probabilistic membership — very useful when cluster/class boundaries genuinely overlap.

Why use GMM: models elliptical (not just spherical) clusters; gives probabilistic membership (useful for uncertainty-aware

downstream decisions); can represent overlapping populations better than hard clustering (K-Means).

19. PCA — Principal Component Analysis

Type: Unsupervised dimensionality reduction — **no class labels used**. Goal: find new axes (principal components) along which the data has **maximum variance**, and project the data onto the top few such axes to reduce dimensionality while retaining as much information (spread) as possible.

19.1 Steps

1. Mean-centre data → **2. Compute covariance matrix** → **3. Eigen-decompose covariance** →
4. Sort eigenvectors by eigenvalue (desc.) → **5. Keep top-k eigenvectors** → **6. Project data onto them**

```
Covariance matrix: Σ = (1/n)·Σᵢ (xᵢ − x̄)(xᵢ − x̄)ᵀ
```

```
Eigen-equation: Σv = λv  (λ = eigenvalue = variance captured along eigenvector v)
```

```
Projection: z = Wᵀ(x − x̄),  where W's columns = top-k eigenvectors sorted by eigenvalue (largest first)
```

```
% variance retained = (Σ of top-k eigenvalues) / (Σ of all eigenvalues) × 100%
```

Worked Numerical — PCA on a small 2-D dataset
Data points: (2,1), (3,3), (4,5), (5,4). Mean $\bar{x}$ = (3.5, 3.25).
Mean-centred data: (−1.5,−2.25), (−0.5,−0.25), (0.5,1.75), (1.5,0.75)
Covariance matrix (illustrative, dividing by n):
$\Sigma xx = [(-1.5)^2+(-0.5)^2+(0.5)^2+(1.5)^2]/4 = [2.25+0.25+0.25+2.25]/4 = 5/4 =$ **1.25**
$\Sigma yy = [(-2.25)^2+(-0.25)^2+(1.75)^2+(0.75)^2]/4 = [5.0625+0.0625+3.0625+0.5625]/4 = 8.75/4 =$ **2.1875**
$\Sigma xy = [(-1.5)(-2.25)+(-0.5)(-0.25)+(0.5)(1.75)+(1.5)(0.75)]/4 = [3.375+0.125+0.875+1.125]/4 = 5.5/4 =$ **1.375**
Σ = [[1.25, 1.375],[1.375, 2.1875]]
Eigenvalues from $\lambda^2 - \text{trace}\cdot\lambda + \det = 0$: trace=3.4375, det=(1.25)(2.1875)−(1.375)²=2.734−1.891=0.843
$\lambda^2 - 3.4375\lambda + 0.843 = 0 \rightarrow \lambda = [3.4375 \pm \sqrt{(11.816-3.373)}]/2 = [3.4375 \pm 2.906]/2 \rightarrow$ **λ1≈3.17, λ2≈0.27**
The eigenvector for λ1 (largest) is the **first principal component** — the direction of maximum spread (roughly along the diagonal, since the data trends from bottom-left to top-right). % variance retained by PC1 alone = 3.17/(3.17+0.27) ≈ **92.2%** — so projecting onto just PC1 keeps most of the information.

Use in computer vision: Eigenfaces (face recognition), compressing SIFT/GLOH descriptors, noise reduction, visualization of high-D feature spaces in 2D/3D.

20. LDA — Linear Discriminant Analysis

Type: Supervised dimensionality reduction — **uses class labels**, unlike PCA. Goal: find a projection that **maximizes the separation between classes** while **minimizing the spread within each class** — directly optimized for classification, not just variance.

20.1 Fisher's Criterion

```
J(w) = (wᵀ·S_B·w) / (wᵀ·S_W·w)
```

```
Within-class scatter: S_W = Σ_c Σ_{x∈class c} (x−μ_c)(x−μ_c)ᵀ
```

```
Between-class scatter: S_B = Σ_c nc·(μ_c−μ)(μ_c−μ)ᵀ   (μ = overall mean, μ_c = class mean, nc = size of class c)
```

Maximizing J(w) is equivalent to solving the generalized eigenvalue problem $S_B\, w = \lambda\, S_W\, w$. For the special **2-class case**, the optimal direction has a closed form:

```
w* = S_W⁻¹ (μ1 − μ2)
```

LDA yields at most **(C−1)** useful discriminant directions for C classes (since S_B has rank at most C−1).

Worked Numerical — 2-Class LDA (1-feature illustration)
Class 1 samples: {2, 3, 4} → μ1 = 3, within-class variance $s1^2 = [(2-3)^2+(3-3)^2+(4-3)^2]/3 = 2/3 \approx 0.667$
Class 2 samples: {8, 9, 10} → μ2 = 9, within-class variance $s2^2 = 2/3 \approx 0.667$
For 1-D data, $S_W \approx s1^2+s2^2 = 1.333$ (sum of within-class scatters); (μ1−μ2) = 3−9 = −6
$w^* \propto S_W^{-1}(\mu1-\mu2) = (1/1.333)\times(-6) \approx$ **−4.5** (in 1-D this is just a scaling factor — the key result is the large gap between class means (6) relative to the small within-class spread (0.667 each), giving a high Fisher ratio J and confirming the classes are well-separated along this feature).
In 2-D/higher dimensions, this generalizes to computing full S_W and S_B matrices, then $w^*=S_W^{-1}(\mu1-\mu2)$ is a vector — project all data as $y=w^Tx$ to get a 1-D discriminant score, then threshold to classify.

20.2 PCA vs LDA (Classic Exam Comparison)

Aspect	PCA	LDA
Supervision	Unsupervised (no labels)	Supervised (uses class labels)
Objective	Maximize total variance	Maximize between-class / within-class scatter ratio
Max useful components	Up to d (original dimension)	Up to C−1 (C = number of classes)
Best used for	Compression, visualization, noise reduction, preprocessing	Classification tasks where labels are available
Combination	PCA is often applied first (to denoise/reduce dimensionality), then LDA is applied for classification — common combined pipeline (PCA+LDA)	

21. Discriminant Functions

Classification via comparing class-specific decision scores.

```
g_k(x) = w_k^T x + w_0k   (linear discriminant function for class k)
```

```
Decision rule: x ∈ ω_k if g_k(x) = max_j g_j(x)   Decision boundary between class i,j: g_i(x) = g_j(x)
```

Types: Linear discriminant → straight-line/hyperplane boundaries; Quadratic discriminant → curved (quadratic) boundaries, needed when class covariances differ significantly; Discriminant analysis can also explicitly model each class's probability distribution (connects to the Bayes classifier below). **Geometric view:** the feature space is divided into decision regions by these boundaries; a good classifier produces low classification error on unseen (test) data, not just on training data.

22. Classifiers: Bayes, KNN, ANN

22.1 Bayes Classifier

Chooses the class with the highest **posterior probability**.

```
P(ω_k | x) = P(x | ω_k)·P(ω_k) / P(x)
```

- $P(\omega_k)$ — prior probability of class k (before seeing data)
- $P(x|\omega_k)$ — likelihood of observing feature x given class k
- $P(\omega_k|x)$ — posterior probability after observing x

```
Bayes decision rule: ŷ = argmax_ω_k P(ω_k | x)
```

Naïve Bayes assumption: features are conditionally independent given the class — this simplifying assumption makes joint probability estimation tractable (P(x|ω) becomes a simple product of per-feature likelihoods).

Worked Numerical — Bayes Classifier
Spam classification: P(spam)=0.4, P(not-spam)=0.6. Given a word "free" appears: P("free"|spam)=0.6, P("free"|not-spam)=0.1.
Unnormalized posteriors: P(spam)·P("free"|spam) = 0.4×0.6 = **0.24**; P(not-spam)·P("free"|not-spam) = 0.6×0.1 = **0.06**.
Since 0.24 > 0.06, the message is classified as **spam** (the exact P(x) normalizing denominator is the same for both and can be skipped when just comparing which posterior is larger).

22.2 K-Nearest Neighbour (KNN)

Instance-based (lazy) classification — no explicit training phase, just stores all training data.

Choose K → **Compute distances to all training points** → **Select K nearest** → **Majority vote** → **Predict class**

```
d(x,q) = √[Σ_i (x_i−q_i)²]      ŷ = mode{y_i : x_i ∈ N_K(q)}
```

Effect of K: small K → sensitive to noise (overfits to local quirks); large K → smoother decision boundary but may blur genuine local structure. **Feature scaling** is essential when dimensions have different units/ranges (else the distance metric is dominated by high-magnitude features).

```
Weighted KNN: w_i = 1/(d_i + ε) (closer neighbours get more influence on the vote)
```

Worked Numerical — KNN
Query point q=(5,5). Training points: A=(1,1,Class1), B=(4,4,Class1), C=(9,9,Class2), D=(6,6,Class2).
Distances: d(q,A)=√(16+16)=5.66; d(q,B)=√(1+1)=1.41; d(q,C)=√(16+16)=5.66; d(q,D)=√(1+1)=1.41.
For K=2, the 2 nearest are B(Class1, d=1.41) and D(Class2, d=1.41) — a tie; with K=3, the third-nearest (among {A,C}) would be checked to break the tie, or weighted-KNN could be used: weight_B≈0.71, weight_D≈0.71 (tie remains — illustrates why choosing an odd K, or using weighted voting with finer continuous weights, matters in practice).

22.3 Artificial Neural Network (ANN)

Learns a **nonlinear** mapping from feature vectors to class probabilities. **Layers:** Input layer → one or more Hidden layer(s) → Output layer. Nonlinear activation functions (ReLU, sigmoid, tanh, etc.) enable complex, non-linear decision boundaries (a

purely linear network, no matter how many layers, could only ever represent a linear boundary).

```
Forward pass (per layer): z = Wx + b,  a = φ(z) (φ = nonlinear activation)
```

Forward pass — compute prediction → **Loss function — measure error** → **Backpropagation — compute gradients** → **Optimizer — update weights**

Backpropagation uses the chain rule to propagate the error gradient backward from the output layer to earlier layers, computing how much each weight contributed to the error; an optimizer (e.g. gradient descent, Adam) then updates every weight to reduce the loss, and this forward→backward→update cycle repeats over many training examples/epochs until convergence.

23. Learning Paradigms and Classifier/Clustering Comparison

23.1 Supervised / Unsupervised / Semi-Supervised

Paradigm	Data	Objective	Examples	When Used
Supervised	Features + labels	Learn f(x) → y	Bayes, KNN, ANN, LDA	Reliable annotations available
Unsupervised	No class labels	Discover structure	K-Means, K-Medoids, GMM, PCA	Exploratory analysis, grouping
Semi-supervised	Small labeled + large unlabeled set	Improve representation/boundary using unlabeled data	Pseudo-labeling, consistency training	Annotation is expensive/limited

23.2 Full Model Comparison Table

Model	Learning	Strength	Limitation	Typical Use
Bayes	Supervised	Principled probabilistic decision	Needs distribution assumptions	Text / medical diagnosis
KNN	Supervised	Simple, uses local structure	Slow prediction, scaling-sensitive	Small/medium datasets
ANN	Supervised	Learns complex nonlinear mappings	Needs lots of data + tuning	Complex classification
K-Means	Unsupervised	Fast clustering	Must choose K	Image grouping
K-Medoids	Unsupervised	Robust to outliers	More expensive	Noisy data clustering
GMM	Unsupervised	Soft, probabilistic, elliptical clusters	Covariance/model selection needed	Overlapping populations
LDA	Supervised	Maximizes class separability	Assumes roughly Gaussian, similar-covariance classes; limited to C−1 dims	Classification with labeled data

23.3 Decision Boundaries — Geometric View

Linear boundary: a line/hyperplane separates classes (linear discriminant, LDA). **Nonlinear classifiers** (ANN, kernel methods) create curved/complex boundaries. **KNN** forms local, piecewise boundaries adapting to nearby data density; **ANN** learns globally optimized nonlinear boundaries; **Bayes** boundaries are determined purely by the underlying class-conditional probability distributions (where posteriors cross: $g_i(x)=g_j(x)$).

23.4 Evaluation of Classifiers and Clustering

```
Confusion matrix terms: TP (true positive), TN (true negative), FP (false positive), FN (false negative)
```

```
Accuracy = (TP+TN)/(TP+TN+FP+FN); Precision = TP/(TP+FP); Recall = TP/(TP+FN); F1 = 2PR/(P+R)
```

Worked Numerical

TP=80, FP=20, FN=10 (and suppose TN=90):

Accuracy = (80+90)/(80+90+20+10) = 170/200 = **0.85**; Precision=**0.80**; Recall≈**0.889**; F1≈**0.842** (as computed earlier in Section 15.3).

For clustering (no ground truth needed): within-cluster sum-of-squares (WCSS), Silhouette coefficient (how well-separated clusters are). **With reference labels available:** Adjusted Rand Index, Normalized Mutual Information.

24. Motion Analysis

24.1 Background Subtraction and Modeling

Used to detect moving foreground objects in a (roughly) static-camera video.

Model the background (e.g. running average / Gaussian per pixel / GMM per pixel) →
Compute |current frame − background model| → **Threshold the difference** → **Foreground mask (moving objects)**

Limitations: sensitive to illumination changes, shadows, dynamic backgrounds (e.g. swaying trees, camera jitter) — more advanced models use a per-pixel Gaussian Mixture (adaptive GMM background model) to handle multi-modal backgrounds.

24.2 Optical Flow

The apparent motion of brightness patterns between consecutive frames, used to estimate per-pixel motion vectors (u,v).

```
Brightness constancy assumption: I(x,y,t) = I(x+u, y+v, t+1)
```

```
Linearized (via Taylor expansion): Ix·u + Iy·v + It = 0  (the "optical flow constraint equation")
```

This is **one equation with two unknowns (u,v)** per pixel — underdetermined ⇒ the **aperture problem** (motion along the edge direction is unobservable from local gradient information alone; only the component of motion perpendicular to a local edge/gradient can be measured). Extra constraints (smoothness, or a local window assumption) are needed to solve for both u and v.

24.3 Lucas-Kanade (KLT) Method

Assumes optical flow is **constant within a small local window** around each pixel — this gives multiple equations (one per pixel in the window) for the same two unknowns (u,v), which can be solved via least squares:

```
Σ[[Ix²,IxIy],[IxIy,Iy²]]·[u,v]ᵀ = −Σ[IxIt, IyIt]ᵀ   (i.e. M·[u,v]ᵀ = b, where M is exactly the Harris structure-tensor matrix!)
```

This connection is a nice exam point: the same 2×2 matrix M used for Harris corner detection also determines the reliability of KLT optical-flow tracking — a good "corner" (both eigenvalues large) is precisely where optical flow can be reliably and uniquely estimated (no aperture problem), which is why KLT trackers pick Harris/Shi-Tomasi corners as the points to track.

24.4 Spatio-Temporal Analysis and Dynamic Stereo

- **Spatio-temporal analysis:** extends single-frame spatial analysis by also considering the time dimension — e.g. treating a video as a 3-D (x,y,t) volume for detecting motion patterns, actions, or events.
- **Dynamic stereo:** extends stereo depth estimation (normally from a static pair of cameras) to a moving scene/camera across time, combining motion cues with multi-view geometry to recover time-varying depth/3-D structure.
- **Motion parameter estimation:** fitting a parametric motion model (e.g. translation, affine, or full projective transform) to observed flow vectors across a region or whole frame — useful for video stabilization, camera-motion compensation, and background-motion separation.

25. Master Formula Sheet & Exam Checklist

25.1 All Key Formulas at a Glance

Topic	Formula
Gradient magnitude/direction	$\|\nabla I\|=\sqrt{(Ix^2+Iy^2)}$; $\theta=\tan^{-1}(Iy/Ix)$
LoG	$\nabla^2 G\sigma = [(x^2+y^2-2\sigma^2)/\sigma^4]\cdot\exp(-(x^2+y^2)/2\sigma^2)$
DoG	$G\sigma1 - G\sigma2$ ($\sigma1<\sigma2$)
Hough (polar)	$\rho = x\cdot\cos\theta + y\cdot\sin\theta$
Harris matrix & response	$M=[[Ix^2,IxIy],[IxIy,Iy^2]]$; $R=\det(M)-k\cdot\text{trace}(M)^2$
Hessian	$\det(H)=Ixx\cdot Iyy-Ixy^2$
Scale-space	$L(x,y,\sigma)=G\sigma*I$; $G\sigma1*G\sigma2=G\sqrt{(\sigma1^2+\sigma2^2)}$
Gabor filter	$g(x,y)=\exp[-(x'^2+\gamma^2y'^2)/2\sigma^2]\cdot\cos(2\pi x'/\lambda+\psi)$
GLCM	Contrast$=\Sigma\Sigma(i-j)^2P(i,j)$; Homogeneity$=\Sigma\Sigma P(i,j)/(1+\|i-j\|)$; Energy$=\Sigma\Sigma P(i,j)^2$
Graph-cut energy	$E(L)=\Sigma D_p(L_p)+\lambda\Sigma w_{pq}[L_p\neq L_q]$
Mean-shift	$x_{(t+1)}=\Sigma x_i\, K(\|x-x_i\|^2/h^2)/\Sigma K(\|x-x_i\|^2/h^2)$
MRF energy	$E(L)=\Sigma\psi(L_p)+\beta\Sigma\psi(L_p,L_q)$
IoU / Precision / Recall / F1	IoU$=\cap/\cup$; P$=$TP/(TP+FP); R$=$TP/(TP+FN); F1$=2PR/(P+R)$
K-Means	$\mu_k=(1/\|C_k\|)\Sigma x_i$; assign to nearest μ
GMM responsibility	$\gamma_k(x)=\pi_k\mathcal{N}(x\|\mu_k,\Sigma_k)/\Sigma_j\pi_j\mathcal{N}(x\|\mu_j,\Sigma_j)$
PCA	$\Sigma=(1/n)\Sigma(x_i-\bar{x})(x_i-\bar{x})^T$; $\Sigma v=\lambda v$; $z=W^T(x-\bar{x})$

LDA (2-class)	$J(w)=w^T S_B w / w^T S_W w$; $w^*=S_W^{-1}(\mu 1-\mu 2)$
Bayes	$P(\omega\|x)=P(x\|\omega)P(\omega)/P(x)$
KNN	$d(x,q)=\sqrt{\Sigma(x_i-q_i)^2}$; weighted $w=1/(d+\varepsilon)$
ANN forward pass	$z=Wx+b$; $a=\varphi(z)$
Optical flow constraint	$Ix\cdot u+Iy\cdot v+It=0$

25.2 Rapid-Recall Checklist Before the Exam

- Sobel Gx,Gy at a pixel → magnitude, direction
- Canny's 5 steps in order + why NMS + why hysteresis
- LoG (exact 2nd derivative) vs DoG (fast approximation) — formulas + zero-crossings
- Hough: derive $\rho=x\cos\theta+y\sin\theta$, build an accumulator table by hand for 2–3 points
- Harris: build M from $Ix^2, Iy^2, IxIy$ → det, trace → R → corner/edge/flat classification
- Moravec vs Harris vs Hessian — one-line differences
- RANSAC's 4-step loop + 3 CV applications (fundamental matrix, matching, stabilization)
- SIFT's full 5-stage pipeline, why 128-D, why normalize+clip
- SURF/HOG/GLOH — detector, descriptor, dimension, invariance, cost (comparison table)
- Which descriptor for medical imaging and why (accuracy over speed → SIFT/GLOH)
- Gaussian/Laplacian pyramid construction + reconstruction formula
- Gabor filter equation + meaning of $\lambda,\theta,\psi,\sigma,\gamma$
- DWT: LL/LH/HL/HH subbands; energy/entropy texture features
- Gabor vs DWT comparison table (localization, cost, redundancy, applications)
- Region growing numerical (seed+threshold walk-through)
- Split-and-merge quadtree steps
- Graph-cut energy formula, data vs smoothness term meaning
- Mean-shift formula + effect of bandwidth h
- MRF energy formula + MAP + optimization methods list
- Full segmentation comparison table (strength/limitation/use per method)
- GLCM contrast/homogeneity/energy numerical
- IoU, Precision, Recall, F1 numericals (practice with different TP/FP/FN)
- K-Means full numerical iteration (assign → update → check convergence)
- K-Medoids vs K-Means differences
- GMM soft assignment concept + EM 2-step loop
- PCA full steps + small numerical (covariance → eigenvalues → variance retained)
- LDA Fisher criterion + $w^*=S_W^{-1}(\mu 1-\mu 2)$ + PCA-vs-LDA table
- Bayes numerical (posterior comparison), Naïve Bayes independence assumption
- KNN numerical (distance table, effect of K, weighted KNN)
- ANN: layers, forward pass equation, backprop 4-step loop
- Supervised/unsupervised/semi-supervised — definitions + examples
- Optical flow equation + aperture problem explanation
- KLT/Lucas-Kanade — same M matrix as Harris, why that matters
- Background subtraction basic idea + limitation